CASE STUDY REPORT

TITLE

Online Shopping System: Database Selection for Enterprise Web Applications

Mahmad Sajid [92400103216]
Shreyansh Bhaliya [92400103292]
Omdeepsinh Jadeja [92400103322]

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026–2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS

5EC6

BATCH

A

CASE STUDY REPORT

RUBRICS FOR CASE STUDY

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	**Problem Analysis, Cloud Concepts & SDG Mapping**	**4 Marks:** Clear problem identification, appropriate cloud concepts, and accurate SDG mapping.	**3 Marks:** Good analysis with minor gaps in concepts or SDG mapping.	**1–2 Marks:** Basic problem understanding with partial cloud concepts or SDG mapping.	**0 Marks:** Inadequate or incorrect analysis and no SDG mapping.	**4**
2	**Security Analysis & Technical Depth**	**4 Marks:** Comprehensive security analysis, well-defined architecture, appropriate technologies, and justified solutions.	**3 Marks:** Good technical explanation with minor omissions.	**1–2 Marks:** Limited security analysis and technical details.	**0 Marks:** No meaningful security analysis or technical explanation.	**4**
3	**Report Quality & References**	**4 Marks:** Well-structured report in IEEE format with comprehensive technical content, proper citations, and relevant references.	**3 Marks:** Good report structure with minor formatting or referencing issues.	**1–2 Marks:** Basic report with limited technical details or incomplete references.	**0 Marks:** Incomplete report with poor documentation and missing references.	**4**
	Total					**12**

Sr. No.	Case Study Title / Problem Statement	COs	Date	R1(4)	R2(4)	R3(4)	Marks (12)	Signature
1	Online Shopping System: Database Selection for Enterprise Web Application	CO4						

Notes / Instructions

1. The case study must be completed in a **group of maximum 3 students**.
2. Students must follow the **given submission deadlines**.
3. The case study report must follow the **provided IEEE Conference Paper Template format.**
4. The final report must be uploaded in **DOCX format on Google Classroom,** and a **hard copy must be submitted** as discussed.
5. The report should include proper **technical analysis, cloud concepts, security aspects, SDG mapping, figures/tables, and IEEE-style references**.
6. Use only **authentic research papers and reliable sources** for references.
7. All group members must contribute equally and be prepared for **presentation and evaluation**.
8. Remove all template instructions/content before final submission.
9. Plagiarism or copied content will not be accepted.

Ms. Priyanka Mangi
Subject Faculty

CASE STUDY REPORT

Online Shopping System: Database Selection for Enterprise Web Applications

Course Name: OPEN SOURCE TECHNOLOGIES (01CE0526)

Co's Mapped - CO4
SDG's Mapped – SDG 9

Abstract—An enterprise online shopping application handles multiple forms of data including customer profiles, product catalogs, orders, invoices, reviews, wish lists, and browsing history. This case study evaluates MySQL, PostgreSQL, and MongoDB using data structure, transaction requirements, relationships, flexibility, scalability, security, licensing, and deployment considerations. The analysis identifies relational data as the core of commerce operations and proposes PostgreSQL as the primary system of record for customers, products, orders, payments, and inventory. MongoDB is considered as an optional store for high-volume, semi-structured browsing or event data, while MySQL remains a viable single-relational-database alternative. A layered architecture, database schema, CRUD operations, deployment strategy, testing plan, security controls, open-source licensing analysis, and SDG 9 mapping are presented. The result is a workload-based database selection approach suitable for an enterprise web application.

Keywords—MySQL, PostgreSQL, MongoDB, SQL, NoSQL, database architecture, e-commerce, open source, database design.

CASE STUDY REPORT

I. INTRODUCTION

Online shopping systems depend on databases for nearly every important business operation. A customer profile must be associated with orders, products must be associated with categories and inventory, and payment records must remain consistent with completed orders. Reviews, wish lists, and browsing history introduce additional data patterns that may be relational, semi-structured, or event-oriented.

The selected university case study asks the student to identify structured and unstructured data, compare SQL and NoSQL, design a database architecture, and justify a suitable database selection for different application modules. The study therefore treats database selection as an engineering decision driven by workload and data characteristics rather than by a single universal ranking.

Open-source database technologies are important in this domain because they provide accessible software, community-driven development, extensive documentation, and freedom to evaluate or adapt technology. They also support practical learning in database administration, security, deployment, and performance engineering.

The objectives of this study are to compare MySQL, PostgreSQL, and MongoDB; map application modules to suitable database models; propose a practical architecture; analyze CRUD, deployment, testing, licensing, and security; and connect the solution with SDG 9, Industry, Innovation and Infrastructure.

II. ORGANIZATION / APPLICATION BACKGROUND

The application considered in this case study is a hypothetical enterprise online shopping platform. It provides customer registration and authentication, product browsing, search, cart management, checkout, order tracking, reviews, wish lists, and browsing-history collection.

The existing software ecosystem can be viewed as a web or mobile client connected to an application/API layer. The application layer implements business rules and communicates with one or more database services. A reverse proxy such as Nginx can terminate TLS and route requests to application instances.

Business requirements include correctness of orders and payments, fast product retrieval, secure customer information, availability during high traffic, backup and recovery, auditable transactions, and maintainable software. The platform should also support future growth without requiring a complete redesign.

Open-source adoption can reduce licensing barriers and provide access to widely used technologies. It also enables students and engineering teams to work with the same database concepts used in production environments.

III. PROBLEM ANALYSIS

The central problem is that one database model may not be equally suitable for every workload. Customer, order, invoice, payment, and inventory records have well-defined relationships and consistency requirements, whereas browsing history and some event data can be high-volume and semi-structured.

Functional requirements include account management, product management, cart operations, checkout, order processing, payment recording, review submission, wish-list management, and browsing-history storage. Non-functional requirements include security, availability, consistency, performance, scalability, backup, recoverability, and maintainability.

Important constraints include limited project resources, the need to use open-source technologies, integration complexity when more than one database is used, and the need to avoid unnecessary operational overhead. The study assumes a conventional enterprise web application with a transactional checkout workflow and growing traffic.

DATA CLASSIFICATION

Data Type	Structure	Typical Workload	Suitable Model
Customer profiles	Structured	CRUD, authentication	Relational
Products & categories	Structured	Search, joins, inventory	Relational
Orders & order items	Highly structured	Transactional	Relational
Invoices & payments	Highly structured	Transactional/audit	Relational
Reviews	Semi-structured	Read/write, text	Relational or document
Wish lists	Structured/semi-structured	CRUD	Relational or document
Browsing history	High-volume event data	Append/read	Document/NoSQL

TABLE I. DATA CHARACTERISTICS AND SUITABLE DATABASE MODELS.

IV. OPEN SOURCE SOLUTION ANALYSIS

MySQL is a relational database with the InnoDB storage engine providing ACID transactions, row-level locking, indexes, foreign keys, and replication capabilities. It is widely used for conventional web applications and is a practical choice when a project prefers a single relational database.

PostgreSQL is an open-source object-relational database that supports SQL, transactions, foreign keys, multiple indexing methods, partitioning, replication, and JSON/JSONB data. These features make it suitable for enterprise workloads that combine strong relational integrity with some semi-structured information.

MongoDB is a document-oriented database using flexible BSON documents. Its document model is useful when

application data is hierarchical or changes shape over time. MongoDB also provides transactions, indexing, replication, and sharding, making it suitable for selected high-volume or flexible-data workloads.

Criterion	MySQL	PostgreSQL	MongoDB
Model	Relational	Object-relational	Document
Schema	Structured	Structured + JSON	Flexible document
Transactions	Strong ACID support	Strong ACID support	ACID incl. multi-document
Relationships	Foreign keys/joins	Foreign keys/joins	Embedding/references
Flexible data	Moderate	High with JSONB	Very high
Horizontal scale	Replication/cluster options	Replication/partitioning	Native sharding
Best fit here	Conventional e-commerce	Core enterprise data	History/events/flexible data
License / model	Community GPL	PostgreSQL License	Community edition licensing varies by version

TABLE II. COMPARISON OF OPEN-SOURCE DATABASE OPTIONS.

For the proposed system, PostgreSQL is selected as the primary database because the core business entities have strong relationships and transactional requirements. Foreign keys, transactions, indexing, partitioning, and JSON support allow the system to keep the core relational model while accommodating selected flexible fields.

The choice is not based on a universal ranking. It is based on the workload described in the case study: transactional commerce data is central, while browsing history and other event-oriented data can be separated when the additional operational complexity is justified. MySQL remains a sound alternative when a simpler single-relational-database deployment is preferred.

V. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

The proposed architecture separates presentation, business logic, database services, and operational services. Users access the application through a web or mobile client. Requests pass through Nginx to application/API instances. The application validates input, applies business rules, and accesses PostgreSQL for authoritative commerce data. MongoDB can be introduced selectively for browsing history or flexible event data.

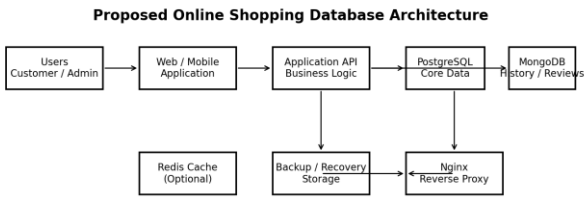


Fig. 1. Proposed online shopping database architecture.

Layer	Suggested Technology	Purpose
Client	HTML/CSS/JavaScript or mobile client	User interaction
Reverse proxy	Nginx	Routing, TLS termination, request handling
Application	Laravel/Django/Node.js	Business logic and APIs
Primary DB	PostgreSQL	Customers, products, orders, payments, inventory
Document DB	MongoDB	Browsing history and selected flexible events
Version control	Git + GitHub/GitLab	Source and schema versioning
Testing	PyTest/PHPUnit/JUnit, Postman, Selenium, JMeter	Quality and performance
Security testing	OWASP ZAP	Web security assessment

TABLE III. PROPOSED TECHNOLOGY STACK.

The database layer should not be exposed directly to the public internet. Application accounts should receive only the privileges required by their functions. Database credentials should be stored securely, connections should use TLS where appropriate, backups should be encrypted, and production changes should be controlled through versioned migrations.

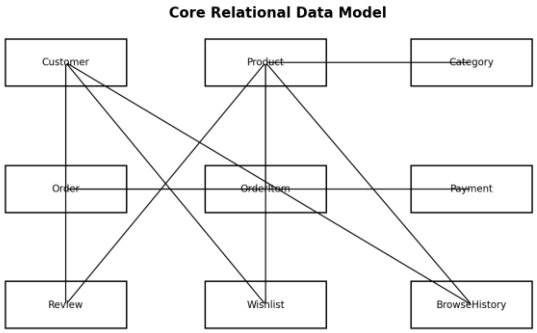
Git with a platform such as GitHub or GitLab can manage source code, database migration scripts, configuration templates, and documentation. A typical workflow is feature branch development, code review, automated tests, merge to the main branch, staging deployment, and controlled production deployment.

VI. DATABASE AND DEPLOYMENT ANALYSIS

The relational schema begins with Customer, Product, Category, Order, OrderItem, Payment, Review, and Wishlist entities. Customer has a one-to-many relationship with orders, while orders contain multiple order items. Products belong to categories and can be referenced by order items, reviews, and wish-list entries.

Table	Important Attributes
Customer	customer_id, name, email, password_hash, address
Category	category_id, name, description
Product	product_id, category_id, name, price, stock_qty
Order	order_id, customer_id, order_date, status, total
OrderItem	order_item_id, order_id, product_id, quantity, unit_price
Payment	payment_id, order_id, method, amount, status, transaction_ref
Review	review_id, customer_id, product_id, rating, review_text, created_at
Wishlist	wishlist_id, customer_id, product_id, created_at

TABLE IV. CORE RELATIONAL DATABASE SCHEMA.



CASE STUDY REPORT

Fig. 2. Core relational data model for the shopping application.

CRUD operations form the core data access pattern. Create operations register customers, products, orders, reviews, and wish-list entries. Read operations retrieve product information, order history, and account data. Update operations change profiles, stock, order states, and reviews where permitted. Delete operations remove or deactivate records according to business and audit rules.

For checkout, the application should use a transaction to create the order, create order items, reduce inventory, and record payment state consistently. If any critical step fails, the transaction should roll back the related database changes. This protects the system from partial order creation.

Deployment should use separate development, testing, and production environments. Database migrations should be version-controlled. Production databases should have scheduled backups, tested restore procedures, monitoring, access controls, and recovery documentation. Nginx can act as a reverse proxy and application load balancer when multiple application instances are deployed.

VII. SOFTWARE QUALITY AND TESTING

Database testing should verify functional correctness, data integrity, security, and performance. Unit tests can validate repository or service methods. API tests can verify authentication, CRUD endpoints, validation, and error handling. Selenium can exercise important browser workflows such as login, product search, cart operations, and checkout.

SQL injection is a major database-related web security concern. OWASP recommends parameterized queries or prepared statements and appropriate input validation. Other controls include least-privilege database accounts, strong credential handling, TLS, secure backups, audit logging, rate limiting, and regular dependency updates.

Performance testing can use Apache JMeter to generate concurrent requests. Important measurements include response time, throughput, error rate, CPU and memory use, database connection utilization, and query latency. Security testing can use OWASP ZAP for web-layer assessment.

Test Area	Example Test	Expected Result
CRUD	Create/update customer	Valid data is stored and returned
Integrity	Order with invalid customer ID	Operation rejected
Transaction	Failure during checkout	Partial changes are rolled back
Security	SQL injection payload	Input is not executed as SQL
API	Invalid authentication token	Unauthorized response
Load	Concurrent product/order requests	Response remains within target
Recovery	Restore database backup	Data is recoverable

TABLE V. SOFTWARE QUALITY AND TESTING PLAN.

VIII. OPEN SOURCE COMMUNITY AND LICENSING ANALYSIS

Open-source licensing affects how database software can be used, modified, and distributed. MySQL Community Edition is distributed under the GPL, while PostgreSQL is distributed under the PostgreSQL License. MongoDB's licensing model should be checked against the exact version and edition selected for deployment.

PostgreSQL's permissive license is designed to allow use, modification, and distribution with relatively few restrictions. MySQL Community Edition's GPL licensing creates different obligations for distribution. Therefore, a project should record the selected edition and version and review the applicable license before redistribution.

Community support is also an important open-source benefit. Official documentation, issue trackers, user communities, extensions, tutorials, and third-party tools can reduce dependence on a single vendor. Version management should use supported releases, documented upgrades, migration testing, and rollback procedures.

IX. TECHNICAL ANALYSIS

The proposed solution uses PostgreSQL as the authoritative store for transactional commerce data and optionally MongoDB for high-volume event-oriented information. The application layer provides a stable interface so that individual database technologies remain behind service boundaries.

Advantages include strong relational integrity for orders and payments, flexible support for semi-structured events, open-source availability, broad tooling, and clear separation of transactional and event workloads.

Limitations include the operational complexity of maintaining two database systems, synchronization and reporting challenges across stores, additional monitoring, and the need for engineers who understand both relational and document databases.

Scalability should be addressed at several levels: application servers can be replicated behind Nginx, PostgreSQL can use indexing, partitioning and replication, and MongoDB can scale selected workloads using replication and sharding. Reliability depends on transactions, backups, tested recovery, monitoring, and controlled deployment.

Maintainability is improved by modular services, schema migrations, documented APIs, automated tests, infrastructure configuration, and version control. Performance analysis should focus on real workload measurements rather than database brand comparisons alone.

X. SDG MAPPING AND JUSTIFICATION

This case study maps to SDG 9: Industry, Innovation and Infrastructure. The goal emphasizes resilient infrastructure, sustainable industrialization, and innovation. An enterprise database architecture is part of digital infrastructure because it supports reliable storage, processing, and access to business information.

SDG 9 Aspect	Case Study Contribution
Innovation	Uses open-source database technologies and flexible data models
Infrastructure	Designs a scalable and recoverable database layer
Technology access	Uses widely available open-source software
Reliability	Applies transactions, backups, replication and testing

CASE STUDY REPORT

Skills	Develops practical database and open-source engineering knowledge

TABLE VI. SDG 9 MAPPING.

The use of open-source databases can lower software acquisition barriers, support local technical learning, and provide access to source code and community knowledge. In an enterprise context, these capabilities can support innovation while maintaining a practical focus on reliability, security, and maintainability.

XI. CHALLENGES AND RECOMMENDATIONS

The main technical challenge is deciding where to place data when multiple database models are available. The project should avoid introducing MongoDB merely because it is a NoSQL database; it should be introduced only where its document model or scaling characteristics provide a clear engineering benefit.

Security challenges include credential exposure, injection attacks, excessive database privileges, insecure backups, unpatched components, and insufficient monitoring. Deployment challenges include configuration consistency, database migrations, backup validation, and maintaining separate development, testing, and production environments.

Future improvements can include read replicas, database partitioning, caching, event streaming, centralized monitoring, automated CI/CD database migrations, and more detailed observability. Recommendations include starting with the simplest architecture that satisfies requirements and adding a second database only when measured workload characteristics justify it.

XII. CONCLUSION

The online shopping case study demonstrates that database selection should be driven by data characteristics and business requirements. Customer, product, order, payment, and inventory data are naturally relational and benefit from transactional integrity, while browsing history and selected event data can be handled with a document-oriented approach when required.

The proposed architecture demonstrates CO4 by applying open-source database architecture principles to a realistic enterprise web application. It also maps to SDG 9 by emphasizing accessible technology, resilient infrastructure, innovation, and practical engineering skills. PostgreSQL is therefore proposed as the primary database for the described workload, with MongoDB as an optional complementary store and MySQL as a viable relational alternative.

REFERENCES

[1] Marwadi University, “Case Study 8: Database Selection for an Online Shopping Website,” Open Source Technologies (01CE0526), Department of Computer Engineering, 2026.

[2] MySQL, “MySQL Community Edition,” Oracle, 2026. [Online]. Available: https://www.mysql.com/products/community/

[3] PostgreSQL Global Development Group, “PostgreSQL: About” and “PostgreSQL License,” 2026. [Online]. Available: https://www.postgresql.org/about/ and https://www.postgresql.org/about/licence/

[4] Oracle, “MySQL 8.4 Reference Manual: Introduction to InnoDB,” 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/innodb-introduction.html

[5] PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2026. [Online]. Available: https://www.postgresql.org/docs/

[6] MongoDB, Inc., “What is MongoDB? Database Manual,” 2026. [Online]. Available: https://www.mongodb.com/docs/manual/

[7] MongoDB, Inc., “Transactions,” MongoDB Database Manual, 2026. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/

[8] OWASP Foundation, “SQL Injection Prevention Cheat Sheet,” OWASP Cheat Sheet Series, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html